

【机器学习模型】

【监督学习】-有标号

- 1、线性回归
- 2、Logistic 回归
- 3、CART（分类与回归树）
- 4、Random Forest(随机森林)
- 5、Boosting Classifier
- 6、KNN (K-Nearest Neighbors, K 近邻算法)
- 7、SVM

【无监督学习】-无标号

- 8、关联性规则
- 9、聚类分析 (K-means、层次聚类)

目录

【机器学习模型】：	1
一、 线性回归 Linear Regression	3
1.1 线性回归模型	3
1.2 线性回归模型（R 代码实现）	4
1.3 线性回归模型诊断检查	4
1.4 分类变量（Categorical X Variables）在回归中的处理	6
1.5 【总结】	8
二、 逻辑回归 Logistic Regression	8
2.1 逻辑函数	8
2.2 几率(Odds)与优势比(Odds Ratio)	10
2.3 混淆矩阵（Confusion Matrix）	13
2.4 总结	14
三、 分类与回归树（CART）	14
3.1 CART 概述	14
3.2 节点纯度、误分类错误与基尼指数(仅适用于分类因变量 Y)	15
3.3 确定复杂度惩罚参数和剪枝	18
3.4 R 代码实现	21
3.5 交叉验证和 1 个标准误差规则	23
3.6 总结	25
四、 随机森林（Random forest）	26
4.1 Bootstrap	26
4.2 随机森林	27
4.2.1 Bootstrap 设计 vs Bagging 设计	27
4.2.2 随机特征子集 (RSF) 选择	29
4.2.3 随机森林 = Bagging + 随机特征子集	30
4.3 R 和 Python 实现	31
4.4 【总结】	32
五、 KNN（K Nearest Neighbor, K 近邻算法）	33
5.1 基本思想	34
5.2 工作流程	34

5.3 应用场景及优缺点	35
5.4 【总结】	36
六、 关联性规则	36
6.1 背景	36
6.2 关联规则	38
6.3 总结	40
七、 聚类分析	40
7.1 概述	40
7.2 层次聚类	41
7.3 K-means 聚类	41
7.4 【K-means 与层次聚类对比】	41
7.5 python 实现	42
7.6 例子	43

一、线性回归 Linear Regression

1.1 线性回归模型

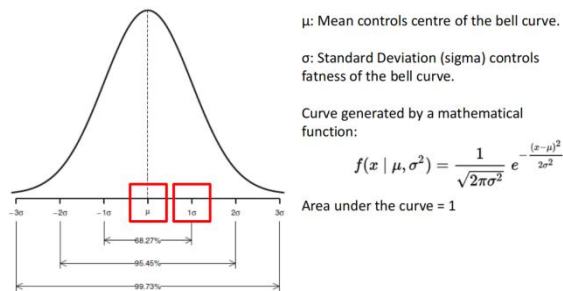
【线性回归模型】

$$y = b_0 + b_1x_1 + b_2x_2 + \cdots + b_mx_m + e$$

其中 $e \sim N(0, \sigma)$ 。

- 直线方程: $\hat{y}$
- 误差 (残差) 服从均值为 0、方差固定的正态分布。
- $y = \hat{y} + e$, 即真实值 = 预测值 + 误差。

Normal Distribution: $X \sim N(\mu, \sigma)$



正态分布

μ : 均值, 控制曲线中心。

σ : 标准差, 控制曲线的“胖瘦”。

曲线由数学函数生成, 总面积 = 1。

【线性回归模型的假设】

$$y = b_0 + b_1x_1 + b_2x_2 + \cdots + b_mx_m + e$$

$\hat{y}$

$e \sim N(0, \sigma)$

- 1、Y 与 X 存在线性关系。
- 2、误差项服从均值为 0 的正态分布。
- 3、误差独立于 X, 且标准差为常数

【线性回归线的解释】

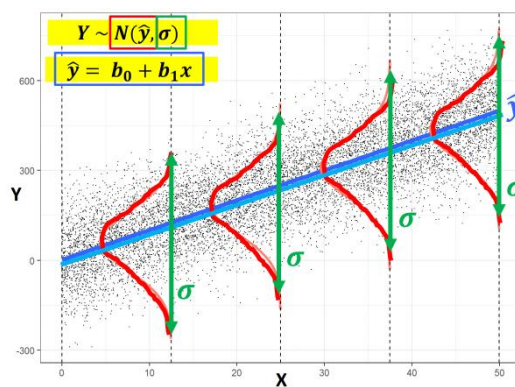
$$y = b_0 + b_1x_1 + b_2x_2 + \cdots + b_mx_m + e$$

$$y = \hat{y} + e, \quad e \sim N(0, \sigma)$$

$$Y \sim N(\hat{y} + 0, \sigma)$$

$$Y \sim N(\hat{y}, \sigma)$$

The straight line $\hat{y}$, represents the mean value of Y (that has a normal distribution) at each value of X s.



$$y = b_0 + b_1x_1 + \cdots + b_mx_m + e, \quad Y \sim N(\hat{y}, \sigma)$$

- 直线 $\hat{y}$ 表示在每个 X 下, Y 的均值 (服从正态分布)。
- $\hat{y}$ = 预测值; $e \sim N(0, \sigma)$ 。

- $Y \sim N(\hat{y}, \sigma)$
- 图中显示: 在直线 $\hat{y}$ 上方, 每个 X 位置的 Y 都围绕 $\hat{y}$ 正态分布。

1.2 线性回归模型 (R 代码实现)

【用 R 得到回归方程】

- 选定因变量 Y (连续型)。
- 确定可能影响 Y 的 X 列表 (若为分类变量, 要设为 factor, R 会自动创建哑变量)。
- 用 `lm()` 拟合线性回归对象。
- 用 `summary()` 查看结果:
- 系数代表每个 X 的斜率。
- $p < 0.05$ 表示统计显著。
- 调整后的 R^2 代表模型拟合优度。
- 用 `plot()` 做诊断图。

模型 m1 的结果

- 回归方程示例:

$$mpg = 37.285 - 5.344 \times wt + e$$

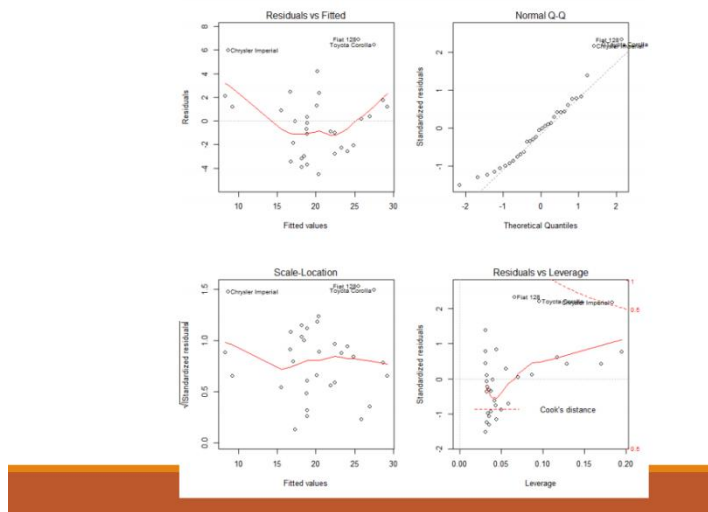
- p 值阈值默认 5% $\rightarrow$ 表示有 5% 的风险“错判关系存在”。
- R^2 : 解释力; 调整 R^2 : 惩罚不必要的变量。

1.3 线性回归模型诊断检查

【模型诊断图】

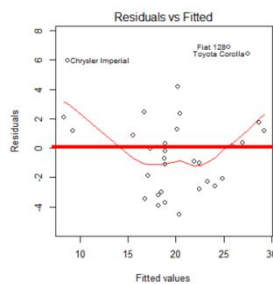
Model Diagnostic Plots

```
> par(mfrow = c(2,2))
> plot(m4)
```



左上：检验 Y 与 X 的线性关系、误差正态性。
 右上 (Q-Q 图)：检验误差是否服从均值 0 的正态分布。
 左下：检验误差是否独立且方差恒定 (同方差性)。
 右下：识别有影响力的异常点。

Top Left Chart



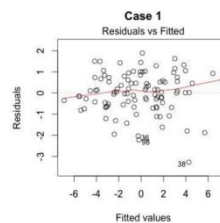
To test

Assumption 1 = Linear Association between Y and Xs.

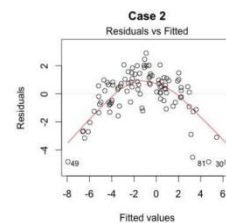
Assumption 2 = Errors has a normal distribution with mean 0.

Top Left Chart (Examples)

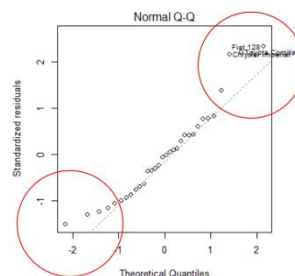
Better Case



Worse Case



Top Right Chart (Q-Q plot)

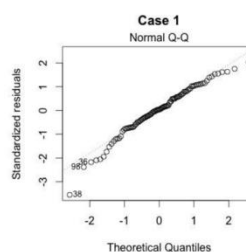


To test

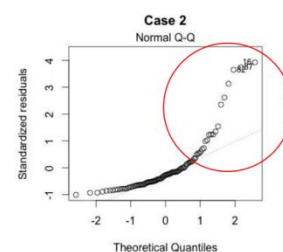
Assumption 2 = Errors has a normal distribution with mean 0.

Top Right Chart (Q-Q plot) (Examples)

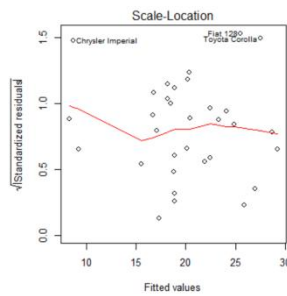
Better Case



Worse Case



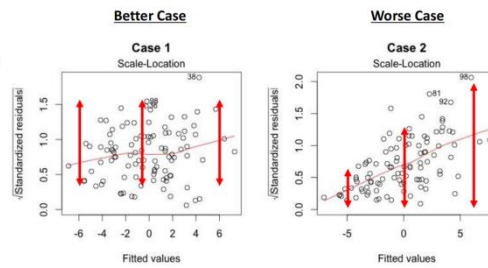
Bottom Left Chart



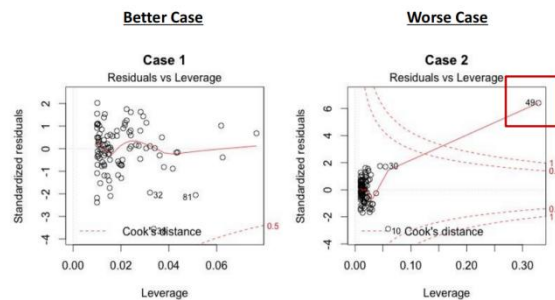
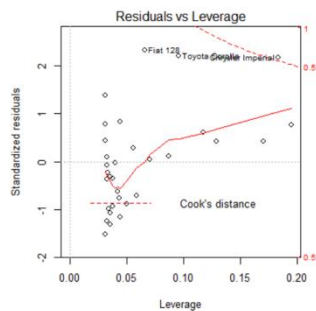
To test

Assumption 3 = Errors are independent of X and has constant standard deviation.

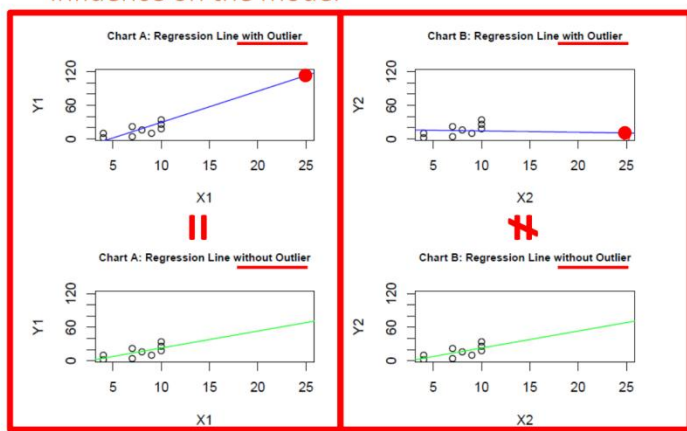
Bottom Left Chart (Examples)



Bottom Right Chart (influential outliers)



Influence on the model



有影响力的异常值

- 异常点分两种：有影响力的 vs 无影响力的。
- 有影响力的异常点会改变模型估计。
- 多变量时无法用散点图直观判断 → 用 Cook's Distance。

1.4 分类变量 (Categorical X Variables) 在回归中的处理

If X is categorical, which model is correct?

- Y: Salary; X: Occupation Code (1: Clerk, 2: Analyst, 3: Manager)
- Linear Regression Model 1 (without dummy variable):
 $\text{Avg.Salary} = 1510 + 700 (\text{occ.code})$
- Linear Regression Model 2 (with dummy variable):
 $\text{Avg.Salary} = 1500 + 465 (\text{occ.code} == 2) + 1156 (\text{occ.code} == 3)$

Note: (occ.code == 1) Group is used as a baseline group.

例：职业代码 (Clerk=1, Analyst=2, Manager=3)。

模型 1 (未处理)：直接代入数字，错误。

模型 2 (哑变量)：设基准组 (occ=1)，再生成 dummy。

R 会自动生成 k-1 个 dummy (基准是字母顺序最小的水平，可用 relevel() 修改)。

示例结果显示不同组的工资/声望分差异。

【如何选择进入回归模型的自变量 (X) ?】

- 专家意见 +
- 领域知识 +
- 统计学意见：
 - X 的 p 值 (小于 5%)。
 - 自动变量选择算法：
 - 后向剔除 (Backward Elimination)
 - 前向选择 (Forward Selection)
 - 双向选择与剔除 (Bidirectional Selection & Elimination)
 - 降维 (特征工程) 方法
 - 用其他模型来选择变量，例如 CRT
 - 其他方法……

【通过方差膨胀因子 (VIF) 检测多重共线性】

如果存在多重共线性的 X 变量：

- 当一个 X 变量可以很好地用其他 X 变量的线性组合来表示时；
- 说明关于该 X 的很多信息已经被其他 X 包含。

- 数学表达：给定模型 M，第 i 个 X 变量 X_i 的 VIF 为：

$$VIF_i = \frac{1}{1 - R_i^2}$$

其中 R_i^2 是以 X_i 为因变量 (Y)，所有其他 X 为自变量所建立的线性回归模型的决定系数。

VIF —— 对临界值没有一致共识

- 一些研究认为：当 $VIF > 5$ （或等价于 $R_i^2 > 0.8$ ）时，存在多重共线性；
- 另一些更严格的研究认为：当 $VIF > 10$ （或等价于 $R_i^2 > 0.9$ ）时，才判定为多重共线性；
- 对含有哑变量的模型：如果 $GVIF > 2$ ，说明可能存在共线性；
- 可以使用 R 外部包 `car` 提供的 `vif()` 函数来计算。

1.5 【总结】

- 1、线性回归 $\neq$ 直线方程本身，还包括假设检验。
- 2、**诊断检查** 是必要环节。
- 3、常见复杂问题：
 - 有影响力的异常点
 - 多重共线性
 - **分类变量（需正确识别为 factor）**

二、逻辑回归 Logistic Regression

- 应用能够预测分类结果变量 Y 的分析模型。
- 识别和量化高风险因素。

2.1 逻辑函数

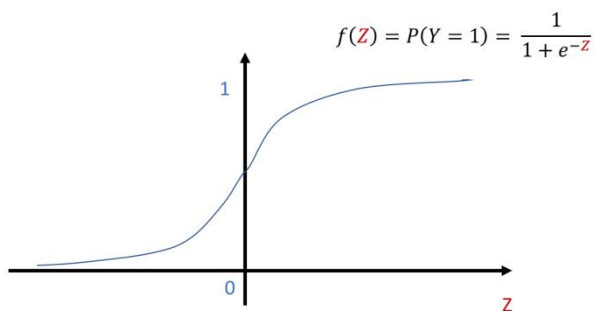
- Y 一定是分类变量（0 or 1/ low or median or high）
- X 不受限制（可以是连续的、也可以是分类的）
- 通过使用 Z 将所有 X 变量组合成一个值
- 概率函数 $P(Y=1)$ 可以作为逻辑函数 $f(x)$
- 输出始终在 0 到 1 之间

Example:

二分类 Y 的逻辑回归模型

$Y=0$ or 1

$$Z = b_0 + b_1X_1 + b_2X_2 + \dots + b_mX_m$$



R 代码实现

Build Logistic Regression Model (Binary Y) in R

- glm() function with family = binomial

- Example:

```
glm(HICOV ~ AGE, family = binomial, data = data1)
```

- Target variable Y is HICOV
 - HICOV must be categorical with two possible categories.
- Input variable is AGE
- Glm is a generalized family of models.
 - Specify binomial to get logistic regression on Binary Y.
- Both variables to be found in R dataframe data1

使用 glm() 函数, 设置 family = binomial

```
library(data.table)
```

```
setwd('D:/Dropbox/Datasets/ADAI/7_Logistic_Reg') # 设置工作目录
```

```
passexam.dt <- fread("passexam.csv") # 读取数据
```

```
passexam.dt$Outcome <- factor(passexam.dt$Outcome) # 将结果转换为因子
```

```
summary(passexam.dt) # 数据摘要
```

```
pass.ml <- glm(Outcome ~ Hours, family = binomial, data = passexam.dt) # 拟合逻辑回归模型
```

```
summary(pass.ml) # 模型摘要
```

Results from summary() function

```
> summary(pass.ml)
Call:
glm(formula = Outcome ~ Hours, family = binomial, data = passexam.dt)

Deviance Residuals:
    Min       1Q   Median       3Q      Max
-1.70557  -0.57357  -0.04654   0.45470   1.82008

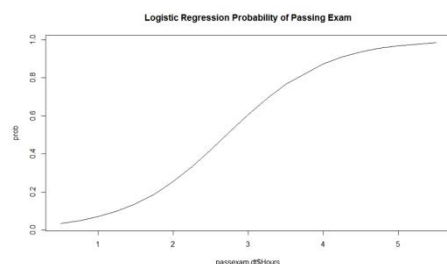
Coefficients:
            Estimate Std. Error z value Pr(>|z|)
(Intercept) -4.0777    1.7610   -2.316   0.0206 *
Hours        1.5046    0.6287    2.393   0.0167 *
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

$$z = -4.0777 + 1.5046(\text{Hours})$$

$$P(Y = 1) = \frac{1}{1 + e^{-z}}$$

Hours is statistically significant factor.

预测值 (P (Y=1))



```
# Export the probabilities from the logistic function for all cases in the data.
prob <- predict(pass.ml, type = "response")
# Now, the y-axis
plot(x = passexam.dt$Hours, y = prob, type = "l", main = "Logistic Regression Probability of Passing Exam")
```

通过比较 $P(Y = 1)$ 与阈值来获取模型预测的 Y (即 $y.hat$)

然后通过比较 $y.hat$ 与实际 Y 得到混淆矩阵 (Confusion Matrix)

```
36 # Set the threshold for predicting Y = 1 based on probability.
37 threshold <- 0.5
38
39 # If probability > threshold, then predict Y = 1, else predict Y = 0.
40 y.hat <- ifelse(prob > threshold, 1, 0)
41
42 # Create a confusion matrix with actuals on rows and predictions on columns.
43 table(passexam.dt$Outcome, y.hat, deparse.level = 2)
44
45 # Overall Accuracy
46 mean(y.hat == passexam.dt$Outcome)
```

```
> table(passexam.dt$Outcome, y.hat, deparse.level = 2)
      y.hat
passexam.dt$Outcome 0 1
0      8 2
1      2 8

> # Overall Accuracy
> mean(y.hat == passexam.dt$Outcome)
[1] 0.8
```

Display Row
name and
Column name

2.2 几率(Odds)与优势比(Odds Ratio)

➤ 事件 A 的几率(Odds)

$$Odds(A) \equiv \frac{P(A)}{1 - P(A)}$$

通常表示为两个数字：整数分子和整数分母。

$P(A)$ 可以是任何概率函数。

Example: Odds of Heart Attack

- Event A: Heart Attack
- If $P(A) = 0.25$, what is the Odds(A)?

$$Odds(A) = 0.25 / (1 - 0.25) = 1/3$$

Odds of A is 1 to 3.

Odds of Event A if $P(A)$ is the logistic function

Let A be the event $Y = 1$.

$$P(A) = P(Y = 1) = \frac{1}{1 + e^{-z}}$$

$$Odds(A) = Odds(Y = 1) \equiv \frac{P(Y = 1)}{1 - P(Y = 1)} = \frac{1}{1 + e^{-z}} \div \frac{e^{-z}}{1 + e^{-z}} = e^z$$

即, $Y=1$ 的几率是线性方程 Z 的指数函数

➤ 如何从 e^z 中分离出模型系数?

$$Odds(Y = 1) = e^z = e^{b_0 + b_1 x_1 + b_2 x_2 + \dots + b_m x_m}$$

- 模型系数 $b_1, b_2, \dots, b_m$ 在 e 的指数内部。
- 要分离它们，回想公式：

$$\frac{a^m}{a^n} = a^{m-n}$$

使用相同底数的分母来消去 m 中你不想要的项。

➤ 连续变量 X_k 的优势比(Odds Ratio)

$$z = b_0 + b_1 X_1 + b_2 X_2 + \dots + b_m X_m$$

$$OR_{X_k}(Y = 1) \equiv \frac{Odds_{X_k+1}(Y = 1)}{Odds_{X_k}(Y = 1)} = e^{b_k}$$

x_k 每增加 1 个单位， $Y = 1$ 的几率乘以 e^{b_k}

如果 $OR > 1$ ，则增加 X_k 将增加 $Y = 1$ 的几率，反之亦然。

Example:

考试通过示例：模型系数 1.5046 的含义是什么？

$$z = -4.0777 + 1.5046(\text{学习小时数})$$

学习小时数是一个连续变量。

$$OR_{\text{学习小时数}}(Y = 1) \equiv \frac{Odds_{\text{学习小时数}+1}(Y = 1)}{Odds_{\text{学习小时数}}(Y = 1)} = e^{1.5046} \approx 4.5$$

每多学习一小时，通过考试的几率将增加 4.5 倍。

➤ 如果 $OR_x(Y=1)=1$ 会怎样？

- X 不影响 $Y=1$ 的几率。
 - 1 是在任何 OR 中都需要关注的基准数字。
 - OR 只是一个分数。
- $OR > 1$ 意味着如果 X 朝特定方向变化，
 - $Y=1$ 的几率将增加。
- $OR < 1$ 意味着如果 X 朝特定方向变化，
 - $Y=1$ 的几率将减少。
- 如果 $OR=0.999876$ 会怎样？— 可以视为 $OR=1$ 吗？
 - 使用 X 的 p 值或 OR 的置信区间来决定
 - 检查 OR 的 95% 置信区间是否包含 1。

Get OR and OR CI from R

```
> OR <- exp(coef(pass.m1))
> OR
(Intercept)      Hours
0.01694617    4.50255687
```

e^{b_1}

```
> OR.CI <- exp(confint(pass.m1))
Waiting for profiling to be done...
> OR.CI
              2.5 %      97.5 %
(Intercept) 0.0001868263 0.2812875
Hours       1.6978380343 23.2228735
```

95% CI excludes 1. Thus, Hours is statistically significant and increasing Hours will increase the odds of passing exam.

Chow C. H.

1

➤ Odds vs Odds Ratio

几率(Odds)

- 为整个线性方程 z 定义。
- e^z
- 是一个函数，因为 z 是一个函数。
- 使用所有属性 $X_1, X_2, \dots, X_m$ 来衡量 $Y = 1$ 的“机会”。

优势比(Odds Ratio)

- 为每个模型系数 b_k 定义。
- e^{b_k}
- 是一个数字，因为 b_k 是一个数字。
- 衡量一个属性 X_k 对 $Y = 1$ 的几率的贡献。

Two Outcomes Y vs Three Outcomes Y

$Y = 0 \text{ or } 1$

$$z = b_0 + b_1X_1 + b_2X_2 + \dots + b_mX_m$$

$$P(Y = 1) = \frac{1}{1 + e^{-z}} = \frac{e^z}{1 + e^z}$$

Denominator to include all Zs.

$$P(Y = 0) = 1 - P(Y = 1)$$

$Y = 0, 1 \text{ or } 2$

$$z_1 = b_{1,0} + b_{1,1}X_1 + b_{1,2}X_2 + \dots + b_{1,m}X_m$$

$$z_2 = b_{2,0} + b_{2,1}X_1 + b_{2,2}X_2 + \dots + b_{2,m}X_m$$

$$P(Y = 1) = \frac{e^{z_1}}{1 + e^{z_1} + e^{z_2}}$$

$$P(Y = 2) = \frac{e^{z_2}}{1 + e^{z_1} + e^{z_2}}$$

$$P(Y = 0) = 1 - P(Y = 1) - P(Y = 2)$$

Two Outcomes Y vs Three Outcomes Y

$Y = 0 \text{ or } 1$	$Y = 0, 1 \text{ or } 2$
$z = b_0 + b_1X_1 + b_2X_2 + \dots + b_mX_m$	$z_1 = b_{1,0} + b_{1,1}X_1 + b_{1,2}X_2 + \dots + b_{1,m}X_m$
	$z_2 = b_{2,0} + b_{2,1}X_1 + b_{2,2}X_2 + \dots + b_{2,m}X_m$
$Odds(Y = 1) \equiv \frac{P(Y = 1)}{1 - P(Y = 1)} \equiv \frac{P(Y = 1)}{P(Y = 0)} = e^z$	$Odds(Y = 1) \equiv \frac{P(Y = 1)}{P(Y = 0)} = e^{z_1}$
	$Odds(Y = 2) \equiv \frac{P(Y = 2)}{P(Y = 0)} = e^{z_2}$

➤ 多分类 Y

Summary for Multicategory Y

Y = 0 or 1	Y = 0, 1, or 2	Y = 0, 1, 2, or 3	Y = 0, 1, 2, ..., k - 1
Y has 2 categories	Y has 3 categories	Y has 4 categories	Y has k categories
1 linear equation z	2 linear equations z ₁ and z ₂ .	3 linear equations z ₁ , z ₂ and z ₃ .	k - 1 linear equations z ₁ , z ₂ and z _{k-1} .
1 Odds e ^z	2 Odds e ^{z₁} and e ^{z₂}	3 Odds e ^{z₁} , e ^{z₂} and e ^{z₃}	k - 1 Odds e ^{z₁} , e ^{z₂} and e ^{z_{k-1}}
2 probabilities	3 probabilities	4 probabilities	k probabilities

2.3 混淆矩阵 (Confusion Matrix)

4. Example of a Confusion Matrix

n=165	Predicted: NO	Predicted: YES
Actual: NO	50	10
Actual: YES	5	100

- A confusion matrix compares model predicted values against the actual data values.
- Refer to Lecture Videos or main textbook Chap 7 for more details and common stats.
- The main diagonal values "50" and "100" rep the number of correct model predictions.
- The other diagonal values "10" and "5" represents the number of wrong model predictions.
- Some research papers publish the overall error or accuracy rate, not confusion matrix.
 - Is this fine?

Metrics calculated from a Confusion Matrix

n=165	Predicted: NO	Predicted: YES
Actual: NO	50	10
Actual: YES	5	100

- True Positive rate = $TP/Actual\ P = 100/105 \approx 0.95$. Aka Sensitivity or Recall.
- False Positive rate = $FP/Actual\ N = 10/60 \approx 0.17$. Aka Type 1 error.
- True Negative rate = $TN/Actual\ N = 50/60 \approx 0.83$. Aka Specificity.
- False Negative rate = $FN/Actual\ P = 5/105 \approx 0.05$. Aka Type 2 error.
- Precision = $TP/Predicted\ P = 100/110 \approx 0.91$.
- Overall Accuracy = $150 / 165 \approx 0.91$; Overall Error = $15 / 165 \approx 0.09$.

2.4 总结

- 逻辑函数用作概率函数。
- Y 变量必须是分类变量。
- X 变量可以是连续的，也可以是分类的。
- 二分类 Y: 使用 Base R 中的 `glm()` 函数。
- 多分类 Y: 使用 `nnet` 包中的 `multinom()` 函数。

三、分类与回归树 (CART)

3.1 CART 概述

- 基本原理
- 构建过程
- 剪枝方法
- 复杂度惩罚
- 节点纯度度量
- 交叉验证与最优树选择

Objective

- 1、理解 CART 模型的基本特征。
- 2、使用 CART 模型：决策规则、预测、总体误差、变量重要性。
- 3、生成 CART 模型：
 - ◆ 阶段 1: 最大化生长树;
 - ◆ 阶段 2: 最小化剪枝 (pruning) ;
 - ◆ 选择最优树
 - 10 折交叉验证 (CV)
 - 1 个标准误差规则

Key Concepts in Phrase 1: Growing Tree to the Maximum

Categorical Y	Continuous Y
- Node: $\hat{Y} = \text{majority}$ - Misclassification Error - Node level - Tree level - Node Purity - Gini Index - Entropy	- Node: $\hat{Y} = \bar{Y}$ - SSE at the node level - MSE at the Tree level
Surrogates	

Key Concepts in Phrase 2: Pruning Tree to the Minimum

- What is pruning?
- Complexity Penalty [Hyperparameter]:
 - Textbook: α
 - Rpart package: `cp`
- Total Cost of using the Model = Model Prediction Error + Complexity Penalty

分类 Y: 节点预测为多数类, 使用误分类误差、Gini 指数、熵。

连续 Y: 节点预测为均值, 使用节点级 SSE、树级 MSE。

代理变量 (surrogates): 处理缺失值

什么是剪枝?

复杂度惩罚 [超参数]:

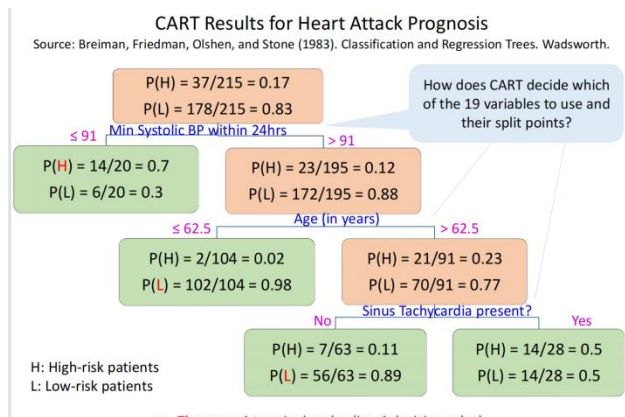
教科书: α

Rpart 包: `cp`

模型使用总成本 = 模型预测误差 + 复杂度惩罚

3.2 节点纯度、误分类错误与基尼指数(仅适用于分类因变量 Y)

Node Purity、Misclassification Error and Gini index



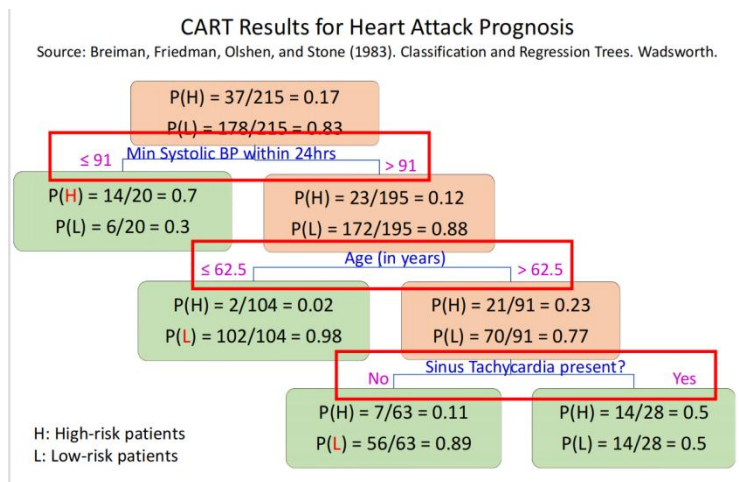
- 共存在 4 个终端节点 terminal node (即 4 条决策规则)
- 终端节点的模型预测基于多数表决原则
- 终端节点的误分类错误 = 少数类比例

【第一阶段关键概念: 树生长至最大化】

Categorical Y
- Node: $\hat{Y} = \text{majority}$ - Misclassification Error - Node level - Tree level - Node Purity - Gini Index - Entropy

选择最佳分裂变量与最佳分裂点

- 在生长阶段的**每个节点**，CART 必须考虑并测试：
 - 所有 X 变量，以及
 - 各 X 变量所有可能取值
 - 以确定**最佳二元分裂**
- 如果一个分裂能产生平均纯度更高的子节点，则该分裂是好的
- 最佳分裂将产生平均纯度尽可能高的子节点
- 理论最纯子节点：某个 Y 类别占比 100%（其他 Y 类别占比 0%）
- 理论最不纯子节点：各个 Y 类别占比相同（即 Y 均匀分布）
- CART 模型预测精度直接受节点纯度影响



- 一个分裂由特定 X 变量和特定取值定义
- 仅使用（并显示）最佳分裂来生成 2 个子节点
- 两个子节点平均纯度更高

为自动化搜索最佳分裂，需用公式指定选择标准

确定最佳二元分裂的公式：

- 基尼指数 (Gini index) ， 或
- 熵 (Entropy)

Leo Breiman 教授偏好使用基尼指数

——包括 Rpart 包在内的多数软件默认设置

误分类错误不是确定最佳二元分裂的好公式

Example: Y = category A or B.

1. Best case: Highest purity occurs when 0% cat A and 100% cat B, or 100% cat A and 0% cat B.
2. Worst case: Highest impurity occurs when 50% cat A and 50% cat B.
3. Preference: Impurity of {40% cat A, 60% cat B} > Impurity of {30% cat A, 70% cat B}
4. Symmetry: Impurity of {40% cat A, 60% cat B} = Impurity of {60% cat A, 40% cat B}

• All formulas used to determine the best split must demonstrate the above results.

熵、基尼指数与误分类错误

令 $p(y|t)$ 表示在树节点 t 中属于 y 类的案例比例

如果 y 只有两种可能结果 $(0, 1)$ ，这三种度量可简化

令 p 代表 $p(y = 1|t)$ ，则 $1-p$ 代表 $p(y = 0|t)$ 。三种不纯度度量变为：

如果 Y 有 $k+1$ 个类别

如果 Y 只有 2 个类别

Entropy, Gini and Misclassification Error

Let $p(y|t)$ denote the proportion of cases belonging to class y at tree node t .

The three measures can be simplified if y has only two possible outcomes $\{0, 1\}$.

Let p represent $p(y = 1|t)$, then $1-p$ represent $p(y = 0|t)$. The 3 impurity measures become:

Entropy(t) =

If Y has $k+1$ categories

If Y has only 2 categories

$$-\sum_{y=0}^k p(y|t) \log_2 p(y|t) = -[p(0|t) \log_2 p(0|t) + p(1|t) \log_2 p(1|t)] = -[(1-p) \log_2 (1-p) + p \log_2 p]$$

Note: $0 \cdot \log(0)$ is defined as 0.

If Y has $k+1$ categories

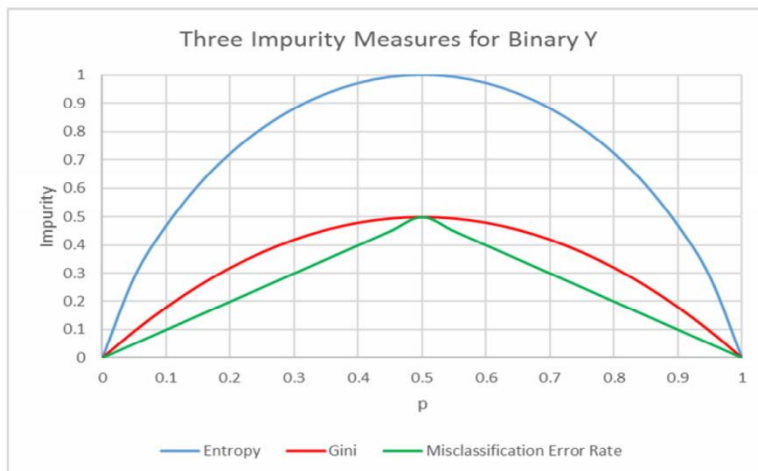
If Y has only 2 categories

$$\text{Gini}(t) = 1 - \sum_{y=0}^k [p(y|t)]^2 = 1 - [p(0|t)^2 + p(1|t)^2] = 1 - [(1-p)^2 + p^2] = 2p(1-p)$$

[Majority Rule] Misclassification error, $r(t) = 1 - \max\{p(y|t)\} = 1 - \max\{p, 1-p\}$

If Y has $k+1$ categories

If Y has only 2 categories



- Gini and Misclassification range from 0 (best) to 0.5 (worst).
- Entropy range from 0 (best) to 1 (worst).

树生长至最大化

- 每一步都寻找最佳分裂变量及其最佳分裂点，并应用于创建两个子节点
- 该过程持续进行，直到满足（宽松的）停止标准
- 结果是一棵非常大的树，很可能过拟合
- 随后需要在第二阶段进行剪枝，将树剪至最小（根节点 root node）
 - 剪枝产生一系列越来越小的树
 - 最优树位于最大树和最小树之间的某处

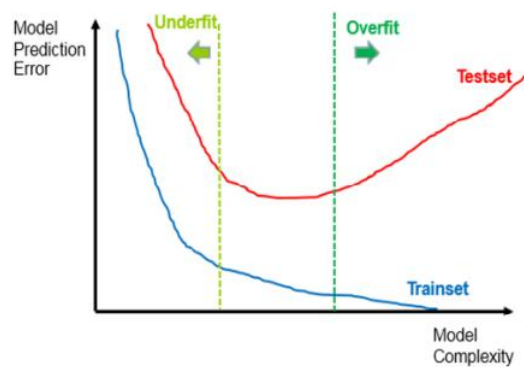
3.3 确定复杂度惩罚参数和剪枝

剪枝的关键在于将复杂度惩罚明确纳入模型使用的总成本中

- 过拟合的风险在所有模型中无处不在。
- 过拟合远比欠拟合更不明显。
- 模型在树生长和树剪枝过程中无法看到测试集。
- 观察过拟合开始时的权衡。
- 过拟合：

■ 随着模型复杂度增加，

- 训练集误差减少，
- 测试集误差增加。



这种权衡可以通过一个公式来体现。

Trade-offs incorporated into a formula via a Complexity Penalty Parameter α

Total Cost of CART = Misclassification Error + Total Complexity Cost

$$R_{\alpha}(T) = R(T) + \alpha |T|$$

Note:

In Breiman (1984) textbook, Complexity of tree $|T|$ = number of terminal nodes.

- As model complexity $|T|$ increases, $R(T)$ decreases but $\alpha \times |T|$ increases.
- α is a positive constant.
- The penalty per unit model complexity.

CART 总成本 = 误分类误差 + 总复杂度成本

$$R_{\alpha}(T) = R(T) + \alpha |T|$$

注意：

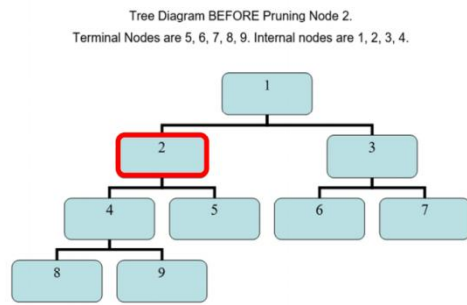
在 Breiman (1984) 的教科书中，树的复杂度 $|T|$ = 终端节点（叶节点）的数量。

- 随着模型复杂度 $|T|$ 增加， $R(T)$ 减少但 $\alpha \times |T|$ 增加。
- α 是一个正常数。
- 每单位模型复杂度的惩罚。

使用复杂度惩罚对最大树进行剪枝

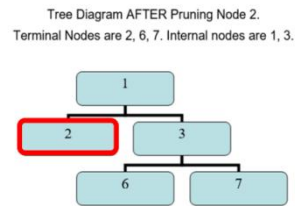
- 给定一棵可能过拟合的大树，需要通过施加树复杂度的惩罚来“修剪”并简化树。
- 惩罚越大，树越小。
- 树的复杂度可以通过以下方式衡量：
 - 终端节点（叶节点）的数量，或
 - 分裂的数量

Pruning the Tree (Before)



• Did you notice the node labelling convention? **Implications?**

Pruning the Tree (After)



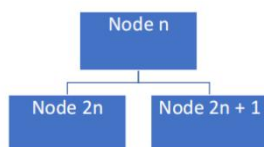
Thus, pruning at node t means all descendants of node t are cut off.
Node t still remains in the tree and becomes a terminal node.

在节点 t 处剪枝意味着节点 t 的所有后代都被剪掉。
节点 t 仍然保留在树中并成为一个终端节点。

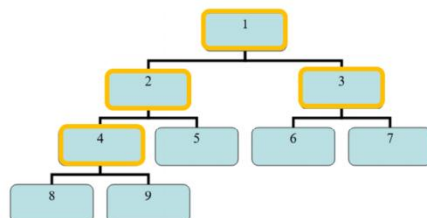
节点标记惯例

- 如果你有一棵大树，这一点尤其重要。
- 大树无法可视化每个节点的细节。
- 通过标准化节点标记惯例，给定任何节点，我们知道：
 - 左子节点是谁（如果有）
 - 右子节点是谁（如果有）
 - 父节点是谁（如果有）

即，无需看到大树即可追溯祖先关系。



Where to prune?



- How to determine the “best” node to prune away?
- Ans: Weakest link pruning.
- How to determine which link is weakest?

- 如何确定要剪掉的“最佳”节点？
 - 答案：最弱链接剪枝。
- 如何确定哪个链接最弱

Total Cost, Adjusted for Model Complexity

Define $\tilde{T}$ to be the set of all **terminal nodes** in the tree T .

Using $r(t)$ and $p(t)$, where $p(t)$ is the proportion of all cases in terminal node t , define the contribution of terminal node t to the total misclassification error as

$$R(t) = r(t) \times p(t) \quad t \in \tilde{T}$$

Overall Misclassification Error of the Tree T :

$$R(T) = \sum_{t \in \tilde{T}} R(t)$$

Complexity Adjusted Total Cost of the Tree T :

$$R_\alpha(T) = R(T) + \alpha |T|$$

Where $|T|$ represents the **size of the tree** e.g. number of terminal nodes, and α represents the **penalty cost per unit complexity**. This is a costing mechanism to penalize complexity. Large α result in small tree.

根据模型复杂度调整后的总成本

定义 $\tilde{T}$ 为树 T 中所有终端节点的集合。

使用 $r(t)$ 和 $p(t)$ ，其中 $p(t)$ 是终端节点 t 中所有案例的比例，将终端节点 t 对总误分类错误的贡献定义为

$$R(t) = r(t) \times p(t) \quad t \in \tilde{T}$$

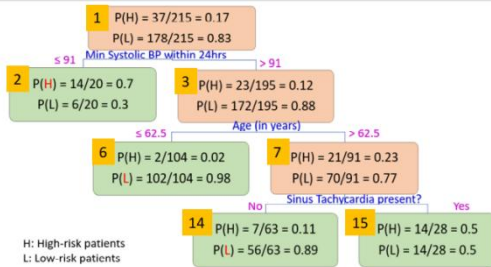
树 T 的整体误分类错误：

$$R(T) = \sum_{t \in \tilde{T}} R(t)$$

树 T 的复杂度调整总成本：

$$R_\alpha(T) = R(T) + \alpha |T|$$

其中 $|T|$ 代表树的大小，例如终端节点的数量， α 代表单位复杂度的惩罚成本。这是一种惩罚复杂度的成本机制。大的 α 会导致小的树。



- $\tilde{T} = \{2, 6, 14, 15\}$
- $r(2) = 0.3, p(2) = 20/215, R(2) = 0.3 \times 20/215 \approx 0.027907$
- $r(6) = 0.02, p(6) = 104/215, R(6) = 0.02 \times 104/215 \approx 0.009674$
- $r(14) = 0.11, p(14) = 63/215, R(14) = 0.11 \times 63/215 \approx 0.03223$
- $r(15) = 0.5, p(15) = 28/215, R(15) = 0.5 \times 28/215 \approx 0.065116$
- $R(T) \approx 0.1349$
- $R_\alpha(T) = 0.1349 + 4\alpha$

Chew C. H.

11

Pruning triggered at certain values of alpha

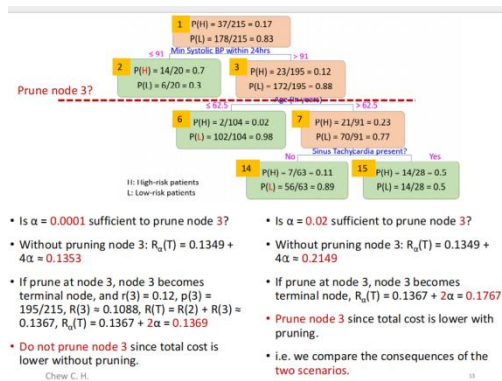
- Only when alpha is big enough will the weakest link be pruned away.
- Chicken Rice Analogy:
 - Current cost in food centre: \$3 per pack. Consume 10 packs per month.
 - What if cost increase to \$3.10 per pack?
 - What if cost increase to \$5 per pack?
 - What if cost increase to \$5.20 per pack?
 - What if cost increase to \$20 per pack?
- CART:
 - Current cost: $\alpha = 0$ units per terminal node. i.e. no penalty for model complexity.
 - What if $\alpha = 0.1$?
 - What if $\alpha = 0.3$?
- As alpha increases, pruning is triggered only at certain few values of alpha, until only the root node is left.

剪枝在特定的 alpha 值处触发

- 只有当 alpha 足够大时，最弱链接才会被剪掉。
- 鸡饭类比 (Chicken Rice Analogy):
 - 食阁当前价格：每包 3 元。每月消费 10 包。
 - 如果成本增加到每包 3.10 元会怎样？
 - 如果成本增加到每包 5 元会怎样？
 - 如果成本增加到每包 5.20 元会怎样？
 - 如果成本增加到每包 20 元会怎样？
- CART:
 - 当前成本：每终端节点 $\alpha=0$ 单位。即，对模型复杂度没有惩罚。
 - 如果 $\alpha=0.1$ 会怎样？

■ 如果 $\alpha=0.3$ 会怎样?

- 随着 α 增加, 剪枝只会在某些特定的 α 值处触发, 直到只剩下根节点。



What minimum value of α will trigger the pruning?

- Compare the two equations from the two scenarios and solve for α .
- Total cost without pruning at node 3 = Total cost if prune at node 3

$$0.1349 + 4\alpha = 0.1367 + 2\alpha$$

$$2\alpha = 0.0018$$

$$\alpha = 0.0009$$

- $\alpha = 0.0009$ is the minimum value of α required to prune at node 3, i.e. the **prune trigger**. [Note that this value is only an approximation as we used only 4 decimal places in the first equation.]

- Any value of α above the minimum value will also prune away node 3.

- Repeat the same analysis at all other internal nodes (i.e. nodes 1, 7) to get their prune triggers.

- The weakest link will be the lowest value among all the prune triggers. E.g. at node X.

- Prune away node X. Recompute $R(T)$, $R_\alpha(T)$, and repeat the analysis at all internal nodes to get the second weakest link, and so on....

- The **pruning sequence** is thus specified and **completely determined by the data**.

- $\alpha=0.0009$ 是在节点 3 处剪枝所需的最小 α 值。即剪枝触发点。[注意, 由于在第一个方程中只使用了 4 位小数, 该值仅为近似值。]
- 任何高于此最小值的 α 值也会剪掉节点 3。
- 在所有其他内部节点 (即节点 1, 7) 重复相同的分析以获得它们的剪枝触发点。
- 最弱链接将是所有剪枝触发点中的最小值。例如在节点 X。
- 剪掉节点 X。重新计算 $R(T)$, $R_\alpha(T)$, 并在所有内部节点重复分析以找到第二弱的链接, 依此类推....
- 因此, 剪枝序列由数据指定并完全确定。

考虑剪枝的另一面, 以计算到后代节点链接的强度

Consider the other side of Pruning in order to compute strength of the link to descendant nodes

- 为了计算剪枝触发点, 我们考虑了从剪枝后留在树中的节点的角度出发, 发生了什么/将会发生什么。
- 我们也可以考虑从被移除节点的角度出发, 发生了什么/将会发生什么。
 - 类比: 杯子满的百分比等同于杯子空的百分比。
- 这种视角将有助于定义到其所有后代的“链接”的强度。
 - 需要额外定义一个以任何内部节点 t 为根的子树, 不一定是以根节点 1 为根。

From the perspective of nodes being considered for removal

Define T_t to be a subtree with root at node t , and define $R_{\alpha}(t) = R(t) + \alpha$

if $R_{\alpha}(T_t) < R_{\alpha}(t)$, then the contribution to the cost complexity of the subtree T_t is less than that for node t (if it becomes a terminal node). Do not prune.

As α increases, equality is achieved when

$$\alpha^* = \frac{R(t) - R(T_t)}{|T_t| - 1}$$

and termination of the tree at t is preferred if $\alpha \geq \alpha^*$ i.e. prune all descendant nodes at node t .

Thus, we can define

$$g(t) = \frac{R(t) - R(T_t)}{|T_t| - 1}$$

as the strength of the link from node t to its descendants.

从考虑被移除的节点的角度出发

定义 T_t 为一个以节点 t 为根的子树, 并定义 $R_{\alpha}(t) = R(t) + \alpha$ (Note: This defines the cost if node t were a terminal node)

如果 $R_{\alpha}(T_t) < R_{\alpha}(t)$, 那么子树 T_t 对成本复杂度的贡献小于节点 t (如果它成为终端节点) 的贡献。不剪枝。

随着 α 增加, 当达到以下等式时:

$$\alpha^* = \frac{R(t) - R(T_t)}{|T_t| - 1}$$

并且在节点 t 终止树是更优的选择, 如果 $\alpha \geq \alpha^*$, 即在节点 t 剪掉所有后代节点。

因此, 我们可以定义

$$g(t) = \frac{R(t) - R(T_t)}{|T_t| - 1}$$

作为从节点 t 到其后代的链接强度。

3.4 R 代码实现

- CART 在 R 包 rpart 中实现。
- 阶段 1 (Phase 1): 生长树至最大。
- 阶段 2 (Phase 2): 剪枝树至最小。
- 剪枝序列确定了一系列子树。
- 其中一个子树是用于 CART 模型预测的最优树。
- 如何使用 CART 模型进行预测。

Differences in the Total Cost Formulas

- Total Cost of CART = Misclassification Error + Total Complexity Cost
- Textbook formula:

$$R_{\alpha}(T) = R(T) + \alpha|T|$$

- $|T|$: Number of terminal nodes.

- rpart formula:

$$R_{cp}(T) = R(T) + cp \times R(T_1) \times |T|$$

- $|T|$: Number of splits. Note: Number of terminal nodes = Number of splits + 1.
- $R(T_1)$: Root Node Error.
- cp : Complexity Parameter. Note: $cp \times R(T_1) = \alpha$

Dataset with categorical Y:

- Predict if Customer will upgrade credit card or not?
- Two X variables:
 - Spending on credit card over last 12 months;
 - Supplementary card – Y/N?

	A	B	C
1	Upgrade	Spending	SuppCard
2	N	8025	N
3	Y	8593	Y
4	N	1219	N
5	N	2032	N
6	N	3245	N
7	N	4012	N
8	N	5166	N
9	Y	10512	Y
10	N	10550	N

```
> summary(custdata1.dt)
Upgrade      Spending      SuppCard
N:18   Min.   :    50   N:17
Y:13   1st Qu.: 6358   Y:14
       Median : 8760
       Mean   : 8405
       3rd Qu.:10550
       Max.   :14804
```

rpart() with method = 'class' for categorical Y

```
18 set.seed(2004) # For randomisation in 10-fold cv.
19
20 # rpart() completes phase 1 & 2 automatically.
21 # Change two default settings in rpart: minsplit and cp.
22 m2 <- rpart(Upgrade ~ Spending + SuppCard, data = custdata1.dt, method = "class",
23            control = rpart.control(minsplit = 2, cp = 0))
```

Default = 20. Changed to 2 due to small sample size. "The minimum number of observations that must exist in a node in order for a split to be attempted." -- ?rpart.control

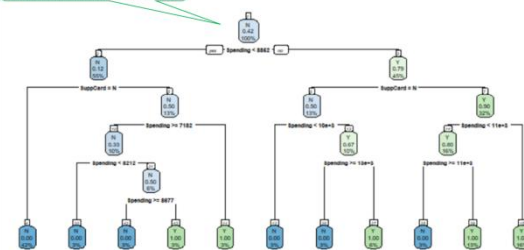
Categorical Y: method = 'class'
Continuous Y: method = 'anova'

Default = 0.01. Changed to 0 to ensure grow tree to the max in phase 1.

Plot the Tree Diagram with rpart.plot()

```
25 # plots the maximal tree and results.
26 rpart.plot(m2, nn=T, main = "Maximal Tree in upgradeCard.csv")
```

In each node, shows node num, $\hat{Y}$, $P(\text{non-baseline } Y)$, $p(t)$.



```
28 # prints the maximal tree m2 onto the console.
29 print(m2)
```

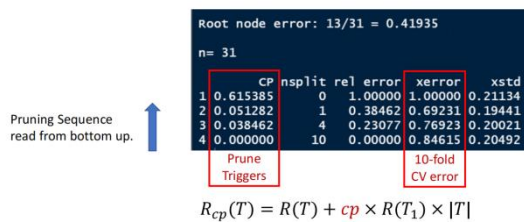
How to read the numbers:

Node number 4:
Decision Rule:
Spending < 8851.5
and SuppCard = No.
Total 13 cases in the
node of which 0
cases will be
misclassified if
predict by majority.
i.e. $\hat{Y} = \text{No}$
100% Y variable = No,
0% Y variable = Yes.
Is a terminal node.

```
> print(m2)
n= 31
node), split, n, loss, yval, (yprob)
* denotes terminal node
1) root 31 13 N (0.5806452 0.4193548)
2) Spending<= 8851.5 17 2 N (0.8823529 0.1176471)
4) SuppCard=N 13 0 N (1.0000000 0.0000000) *
5) SuppCard=Y 4 2 N (0.5000000 0.5000000)
10) Spending<= 8212.1 0 N (0.6666667 0.3333333)
20) Spending<= 8212.1 0 N (1.0000000 0.0000000) *
21) Spending<= 8212.2 1 N (0.5000000 0.5000000)
42) Spending<= 8676.5 1 0 N (1.0000000 0.0000000) *
43) Spending<= 8676.5 1 0 Y (0.0000000 1.0000000) *
11) Spending<= 7182.1 0 Y (0.0000000 1.0000000) *
3) Spending<= 8851.5 14 3 Y (0.2142857 0.7857143)
6) SuppCard=N 4 2 N (0.5000000 0.5000000)
12) Spending<= 10120.1 0 N (1.0000000 0.0000000) *
13) Spending<= 10120.3 1 Y (0.3333333 0.6666667)
26) Spending<= 12893.5 1 0 N (1.0000000 0.0000000) *
27) Spending<= 12893.5 2 0 Y (0.0000000 1.0000000) *
7) SuppCard=Y 10 1 Y (0.1000000 0.9000000)
14) Spending<= 11496.5 1 Y (0.2000000 0.8000000)
28) Spending<= 10534.5 1 0 N (1.0000000 0.0000000) *
29) Spending<= 10534.5 4 0 Y (0.0000000 1.0000000) *
15) Spending<= 11496.5 0 Y (0.0000000 1.0000000) *
```

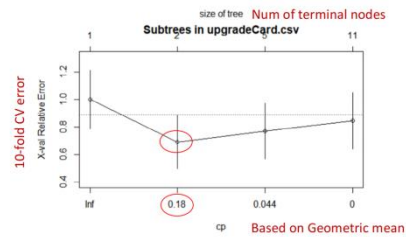
Prune Triggers and Pruning Sequence with printcp().
Can be used to identify optimal tree.

```
31 # prints out the pruning sequence and 10-fold cv errors, as a table.
32 printcp(m2)
```



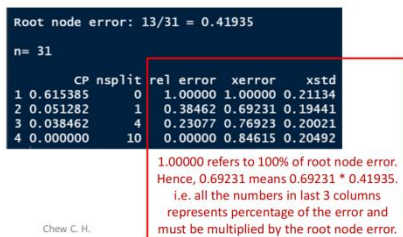
Plot subtrees from the pruning sequence with plotcp(). Can be used to identify optimal tree.

```
34 # Display the pruning sequence and 10-fold cv errors, as a chart.
35 plotcp(m2, main = "Subtrees in upgradeCard.csv")
```



All errors in CP table are scaled against the Root Node Error

```
31 # prints out the pruning sequence and 10-fold cv errors, as a table.
32 printcp(m2)
```

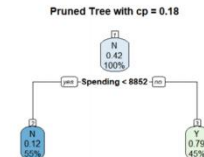


Chew C. H.

```
41 cp1 <- 0.18
42
43 m3 <- prune(m2, cp = cp1)
44
45 printcp(m3)
46
47 # plots the tree m3 pruned using cp1.
48 rpart.plot(m3, nn= T, main = "Pruned Tree with cp = 0.18")
```

Root node error: 13/31 = 0.41935

	CP	nsplit	rel error	xerror	xstd
1	0.61538	0	1.00000	1.00000	0.21134
2	0.18000	1	0.38462	0.69231	0.19441



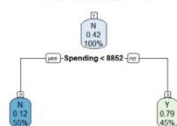
CART Model Predictions

CART can auto-handle missing values
in trainset, testset or what-ifs.

```
50 # Test CART model m3 predictions
51 testcases <- data.frame(Spending = c(8000, 10000, NA), SuppCard = c("Y", NA, NA))
52
53 cart.predict <- predict(m3, newdata = testcases, type = "class")
54
55 results <- data.frame(testcases, cart.predict)
```

	Spending	SuppCard	cart.predict
1	8000	Y	N
2	10000	NA	Y
3	NA	NA	N

Pruned Tree with cp = 0.18



3.5 交叉验证和 1 个标准误差规则

- CART 如何自动处理缺失值 - 代理分裂 (Surrogates)。
- 一种识别最优树的更好方法。
 - 具有最小交叉验证误差的子树是一个不稳定的解。
- 10 折交叉验证 (10 fold cross validation)。

CP Table shows min CV error and its 1SE.

```
31 # prints out the pruning sequence and 10-fold CV errors, as a table.
32 printcp(m2)
```

```
Root node error: 13/31 = 0.41935
n= 31
      cp nsplit rel error xerror xstd
1 0.615385    0  1.00000 1.00000 0.21134
2 0.051282    1  0.38462 0.69231 0.19441
3 0.038462    4  0.23077 0.76923 0.20021
4 0.000000   10  0.00000 0.84615 0.20492
```

Geometric mean CP of the 2nd Tree = $\sqrt{0.0512 \cdot 0.615} \approx 0.18$

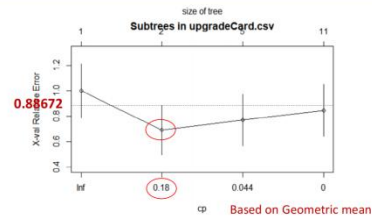
Min CV Error = 0.69231

1SE = 0.19441

CV Error Cap = 0.69231 + 0.19441 = **0.88672**

CV error cap is displayed in plotcp()

```
34 # Display the pruning sequence and 10-fold CV errors, as a chart.
35 plotcp(m2, main = "Subtrees in upgradeCard.csv")
```

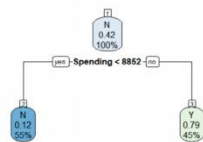


Get a specific subtree via prune() by pruning the maximal tree m2 with a specific value of cp

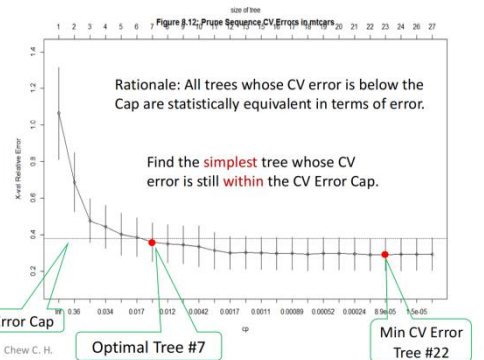
```
41 cp1 <- 0.18
42
43 m3 <- prune(m2, cp = cp1)
44
45 printcp(m3)
46
47 # plots the tree m3 pruned using cp1.
48 rpart.plot(m3, nn=T, main = "Pruned Tree with cp = 0.18")
```

```
Root node error: 13/31 = 0.41935
n= 31
      cp nsplit rel error xerror xstd
1 0.61538 0  1.00000 1.00000 0.21134
2 0.18000 1  0.38462 0.69231 0.19441
```

Pruned Tree with cp = 0.18



CART on mtcars dataset shows 25 trees



Trainset error vs 10 fold Cross Validation (CV) error

```
31 # prints out the pruning sequence and 10-fold CV errors, as a table.
32 printcp(m2)
```

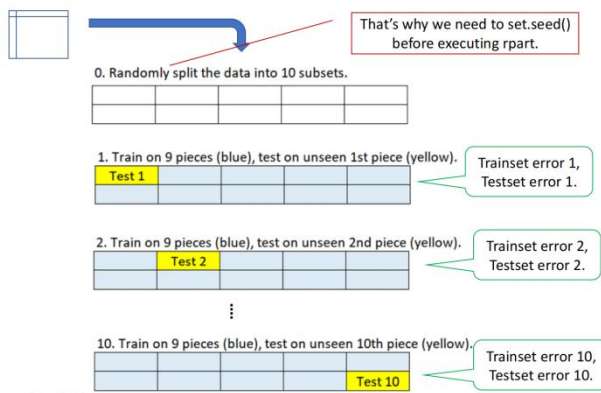
```
Root node error: 13/31 = 0.41935
n= 31
      cp nsplit rel error xerror xstd
1 0.615385    0  1.00000 1.00000 0.21134
2 0.051282    1  0.38462 0.69231 0.19441
3 0.038462    4  0.23077 0.76923 0.20021
4 0.000000   10  0.00000 0.84615 0.20492
```

Trainset error

10-fold CV error

Plotted as the y-coor in plotcp() chart.

What is 10-fold Cross Validation Error?



1 个标准误差规则只是选择最优树的一个指导原则。

● 最小 CV 误差树是一个不稳定的解决方案。

- 数据的微小变化可能导致不同的解决方案。
- 取决于 10 折 CV 中随机的子集划分。
- 1 个标准误差规则更稳定。
 - 许多树在误差方面是统计等价的。
 - 选择仍然表现良好且最简单的树。

3.6 总结

CART 的方法总结：

- 类别型 Y vs 连续型 Y

类型	类别型 Y	连续型 Y
<code>rpart()</code> 参数	<code>method="class"</code>	<code>method="anova"</code>
预测结果	多数类	Y 的均值
最佳分裂标准	基尼指数 (Gini index)	SSE (平方误差和)
性能评估指标	误分类率	MSE

CART 总结：

- 阶段 1：把树尽量长地生长
- 阶段 2：通过剪枝得到最小树
- 使用 10 折交叉验证 + 1SE 规则选择最优树
- 在终端节点提取决策规则（即预测值）
- 变量重要性
- CART 可通过替代分裂 (Surrogates) 自动处理缺失值

四、随机森林 (Random forest)

4.1 Bootstrap

—— 一种采样方法，而非模型 (随机有放回抽样)

—— 随机森林中的一个关键技术

- Bootstrap 是一种通用的统计技术 (采样方法)，**而不是模型**。
- 它是一种极具创造性的方式，来使用已有的数据样本。
- 可以在**不依赖传统假设**的情况下，从样本推断到未知总体。
- 是一些高级机器学习方法的基础
- 随机森林就必须依赖

【从样本到总体的推断 (不用 bootstrap)】

样本 vs 总体：样本统计量已知，总体参数未知。

置信区间公式通常依赖 CLT 或正态分布假设。

比如均值、标准差、比例、相关系数的区间推断，都有特定的前提假设 (如正态、独立性)。

Inference from Sample to Population (without bootstrap)

Population Parameter	Sample Statistic	Inference via Confidence Interval	Key Inference Assumption
Mean: μ	$\bar{x}$	$\bar{x} \pm z^* \frac{\sigma}{\sqrt{n}}$ or $\bar{x} \pm t^* \frac{s}{\sqrt{n}}$	CLT applies (with sufficient sample size), or x has normal dist.
Standard Dev: σ	s	$\sqrt{\frac{(n-1)s^2}{\chi_{df, \frac{\alpha}{2}}^2}} < \sigma < \sqrt{\frac{(n-1)s^2}{\chi_{df, 1-\frac{\alpha}{2}}^2}}$	x has normal dist.

For 95% CI: $z^* = 1.96$, $t^* = 2.776$, $\chi^2_{df=11, 0.975} = 23.65$, $\chi^2_{df=11, 0.025} = 58.12$

Inference from Sample to Population (without bootstrap)

Population Parameter	Sample Statistic	Inference via Confidence Interval	Key Inference Assumption
Proportion: π	p	$p \pm z^* \sqrt{\frac{p(1-p)}{n}}$	π is a binomial dist. parameter & $n\pi \geq 5$, $n(1-\pi) \geq 5$
Correlation: ρ	r	Fisher Transform: $F = \frac{1}{2} \ln \frac{1+r}{1-r}$ $F_l = F - \frac{z^*}{\sqrt{n-3}}$ $F_u = F + \frac{z^*}{\sqrt{n-3}}$ $\frac{e^{2F_l} - 1}{e^{2F_l} + 1} < \rho < \frac{e^{2F_u} - 1}{e^{2F_u} + 1}$	x_i and x_j have iid Bivariate Normal Dist.

【基于模型的推断】

线性回归：回归系数的 P 值与置信区间。

逻辑回归：系数、比值比 (Odds Ratio) 的 P 值与置信区间。

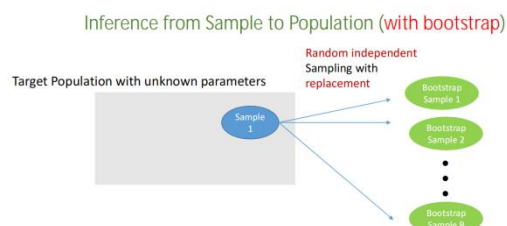
依赖 t、z 或 χ^2 分布。

【非标准推断】

当假设不成立 (非独立、非正态等)，或参数没有标准公式 (如中位数、百分位数、裁剪均值)，如何推断？

答案：Bootstrap。

【从样本到总体的推断 (使用 bootstrap)】



- 每个 bootstrap 样本与原始样本大小相同 (n)。
- B 选为较大数，例如 2000、10000 等。——» 当 B 足够大时，没被选中的样本的概率为 33%

- 基于 B 个 bootstrap 样本来推断总体参数。

【用 Python 做 bootstrap】

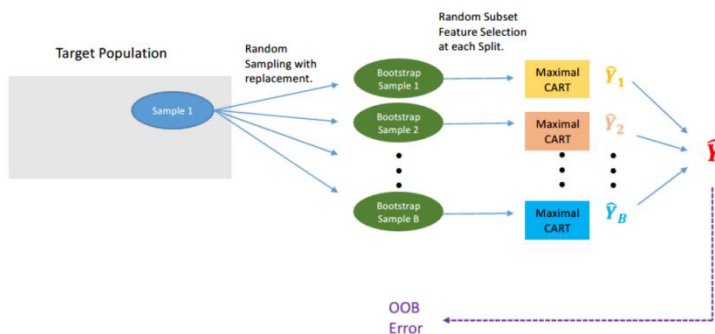
- scikit-learn 的 `resample()` 函数
- 百分位法置信区间：
<https://machinelearningmastery.com/calculate-bootstrap-confidence-intervals-machine-learning-results-python/>
- Facebook 出品的 bootstrapped 包

总结

- Bootstrap = 随机有放回抽样。
- 通常 $B = 10000$ 。
- R 的 boot 包：
成本：你需要写出统计量函数。
好处：你能对任意统计量做 bootstrap，获得分布用于推断。
- 多种 CI 方法可选，推荐 BCA。
- 随机森林使用 bootstrap 来提升 CART 的稳定性

4.2 随机森林

Random Forest as a Process

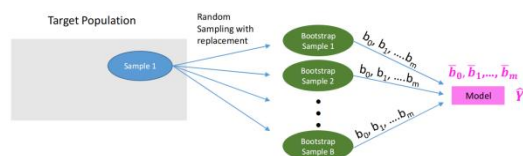


4.2.1 Bootstrap 设计 vs Bagging 设计

Bootstrap → 数据层面的采样技术。

Bagging → 使用 bootstrap 的多个采样集，训练 B 个模型并集成。

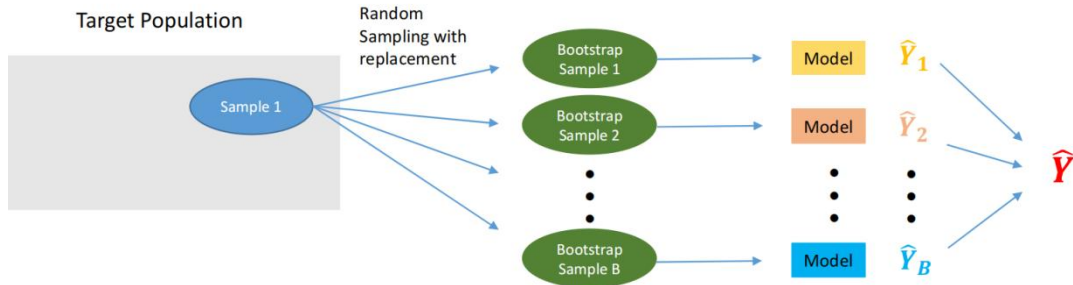
【线性回归模型的 Bootstrap 设计】



- 通过有放回的随机抽样生成 B 个 bootstrap 样本

- 每个 bootstrap 样本会得到一组不同的模型系数 $b_0, b_1, \dots, b_m$ 。
- 取这些系数的平均值作为单个线性回归模型的 bootstrap 估计。
- 然后用这个模型来做预测（得到一个预测值）。
- 这种方法对任何模型类型都适用。

【Bootstrap 聚合 (Bagging)】



- 每个 bootstrap 样本大小与原始样本相同 (n)，生成 B 个 bootstrap 样本
- 每个 bootstrap 样本用来训练同一类型的模型（如 CART）（与前面的 Bootstrap 设计不同）——» 可以用任意模型（如线性回归、逻辑回归），但最常见是 CART。
- Bootstrap 样本数 = 模型数。
- 最终得到 B 个预测，然后聚合它们作为最终预测。
- Breiman (1996a) 的实验：
bootstrap 样本是否要比原样本更大？
 B 需要取多大才合理？
- 每个 bootstrap 样本大小与原始样本相同 (n)。
如果翻倍到 $2n$ ，准确率不会提升 (Breiman, 1996a)。
- B 通常取一个较大的值：分类树：50；回归树：25
这不是必要或充分条件，只是经验值。
实践中， B 需要足够大，直到错误率趋于稳定。
- 测试误差来自每个 bootstrap 的 OOB 样本，取平均。

Bagging 的预测方式

连续型 Y ：取 B 个预测的平均值。

分类 Y ：取 B 个预测的众数（多数表决）。

Bagging 的误差

训练集误差：取 B 个 bootstrap 样本的平均误差。

测试集误差：

Breiman (1996a) 用 90-10 划分，重复 100 次。

后来在 Breiman (2001) 提出了更自然高效的方法 —— OOB 误差。

【OOB 误差】

每个 bootstrap 样本天然会留下 Out-of-Bag (OOB) 样本。

OOB 样本可作为该 bootstrap 样本的测试集。

OOB 误差作为 Bagging 的测试误差

对于大小为 n 的 bootstrap:

某个观测 i 未被选中的概率: $P(\text{OOB}) = (1 - 1/n)^n$

当 $n=10$, ≈ 0.3487

当 $n \rightarrow \infty$, 趋近于 $1/e \approx 0.368 \approx 1/3$

所以约三分之一的原始样本不会出现在 bootstrap 样本中, 可以作为测试集。

因此, OOB 样本误差就是 Bagging 的自然测试误差。

ISLR 总结: “每棵 bagged 树大约使用了 $2/3$ 的观测值。”

Bagging 的表现:

在哪些情况下效果好

在哪些情况下精度没有提升

引出随机森林的改进……

4.2.2 随机特征子集 (RSF) 选择

【模型的稳定性】

稳定性: 指建模过程中结果的一致性, 例如线性回归通过最小 SSE 得到系数。

Bagging 在“不稳定的过程”中效果更好:

- 小的训练集变化可能导致模型有较大变化。

不稳定的例子: 神经网络、CART、MARS、线性回归中的变量选择。

稳定的例子: KNN、岭回归。

【关于模型稳定性的关键学习点】

Bagging 对不稳定过程有效 $\rightarrow$ 起到“稳定化”的作用。

Bagging 对稳定过程无效。

如果所有模型几乎相同, Bagging 没有额外好处。

程序稳定性是相对

线性回归本身是“不稳定的”。

但“包含所有 X ”的线性回归比“变量选择”后的线性回归更稳定。

即: 如果要挑选进入回归的自变量, 过程会更不稳定。

【例】

主导性变量对稳定性的影响

示例: 预测课程成绩 (Y)

X_1 = 学习时间

X_2 = 统计考试分数

X_3 = 课程总评成绩 (占比 70%)

$\rightarrow X_3$ 是主导变量。

任何预测 Y 的模型都会使用 X_3 。

因此各个 bootstrap 模型变得相似, Bagging 难以提升效果。

Bagging 无效的两个原因:

- 稳定的建模过程

● 主导性变量存在

原因：不同 bootstrap 样本生成的模型几乎一样。

如何提高集成方法（如 Bagging）的结果？

如何在存在主导因素 X 的情况下引入可控的不稳定因素？

- 使用更不稳定的方法会带来风险：可能会增加错误发生率。
- 可控的不稳定：在保持准确性的同时增加模型的多样性。

改进思路：

引入“受控的不稳定性”，即在保持精度的同时增加模型多样性。

做法：

在每次树的分裂时，随机选择一部分 X 特征作为候选（RSF）。

每棵树都从 bootstrap 样本中完全生长到最大（不剪枝），确保训练精度

【随机特征子集（RSF）的作用】

- 在树的每次分裂中，只考虑随机挑选的一部分自变量 (X)，用它们来寻找最佳切分点。
- 其他未被选中的 X 在该节点没有机会被选为切分点。
- 在下一个分裂时，重新随机选择新的 X 子集。

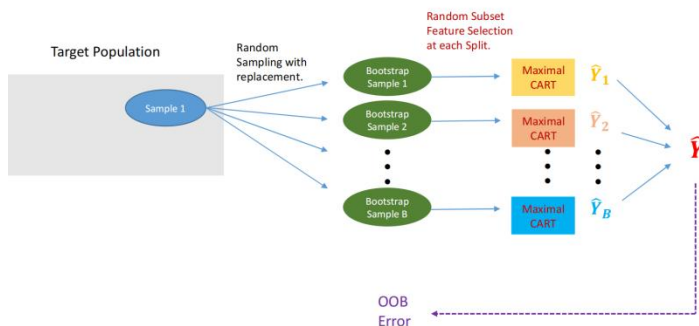
效果：

引入不稳定性 → 不同树会用不同的 X 子集。

防止某个主导变量在所有树中都占据主导，从而避免模型高度相似。

4.2.3 随机森林 = Bagging + 随机特征子集

Random Forest Process



【随机森林流程】

- 从原始样本生成 B 个 bootstrap 样本。
- 每个样本生长一棵最大化的 CART 树（不剪枝）。
- 在每次分裂时使用随机特征子集选择（RSF）。
- 使用 OOB 误差评估性能。

Random Forest 由两个关键思想构成：

Bagging (Bootstrap Aggregation)

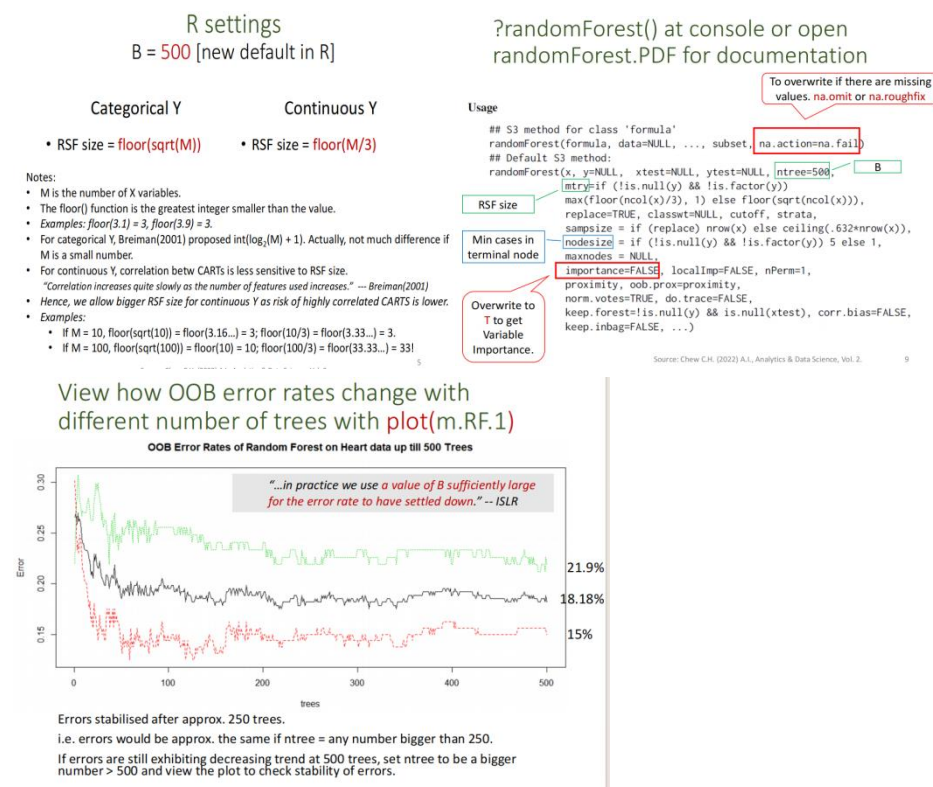
- 用 Bootstrap 抽样 从训练集产生多个子样本集；
- 在这些子样本集上分别训练多个决策树；
- 最终通过投票（分类）或平均（回归）来得到结果，降低方差。

Random Subset Feature Selection (随机特征子集选择)——降低树的相关性

- 在每个树的节点分裂时，只从所有特征中随机抽取一个子集，
- 再从这个子集中选择最佳分裂特征；
- 这样可以降低不同树之间的相关性，使模型更鲁棒。

4.3 R 和 Python 实现

【R 代码实现】



在 R 控制台输入 `?randomForest()`

或者打开 `randomForest.PDF` 文档

缺失值处理: `na.omit` 或 `na.roughfix`

设置 `importance=T` → 输出变量重要性

其他参数: B、RSF 大小、叶子最小样本数等

OOB 误差随树数变化

使用 `plot(m.RF.1)` 查看 OOB 误差曲线。

大约在 250 棵树后误差趋于稳定。

如果 500 棵时误差还在下降，说明 B 需要设得更大。

ISLR 建议: B 要足够大直到误差稳定。

查看预测结果

`m.RF.1$predicted` → 每个样本的 RF 预测值

查看 OOB 次数

m.RF.1\$oob.times → 每个样本在 500 棵树中作为 OOB 的次数

理论值: $P(\text{OOB}) = (1-1/n)^n$

$n=297 \rightarrow$ 约 0.367

案例:

Case1: $177/500=0.354$

Case2: $185/500=0.37$

查看 In-bag / OOB

设置 keep.inbag=T → 可检查每个样本在每棵树中是否在袋内。

查看投票结果

m.RF.1\$votes → 每个样本的投票分布

注意: 只统计样本作为 OOB 时的投票, 不是所有 500 棵树。

注意事项

m.RF.1\$err.rate 不是每棵树的误差率, 而是累积前 k 棵树的 OOB 误差。

例如: Row4=0.27 → 意味着前 4 棵树的 OOB 误差=27%。

【Python 实现】

● Python 实现 (分类 Y)

sklearn.ensemble.RandomForestClassifier

默认参数:

B=100 (需改为 500)

RSF = $\sqrt{M}$ (与 R 一致)

● Python 实现 (连续 Y)

sklearn.ensemble.RandomForestRegressor

默认参数:

B=100 (需改为 500)

RSF = M (需改为 $M/3$)

4.4 【总结】

- 随机森林 = Bagging + 随机特征子集
- Bagging
 - B=500 bootstrap 样本
 - 每个样本长最大化的 CART 树
- RSF 默认:
 - 连续 Y → $\text{floor}(M/3)$
 - 分类 Y → $\text{floor}(\sqrt{M})$
- 误差来自 OOB 样本
- 检查误差是否在 500 棵树前就收敛
- 随机森林提供变量重要性度量

Python RF Implementation

- Categorical Y:
 - `sklearn.ensemble.RandomForestClassifier`
 - <https://scikit-learn.org/stable/modules/generated/sklearn.ensemble.RandomForestClassifier.html>
- Compared to R `randomForest()`:
 - Python used default `B = 100`. Please overwrite to 500.
 - Python also used default RSF size = `int(sqrt(M))`.

Python RF Implementation

- Continuous Y:
 - `sklearn.ensemble.RandomForestRegressor`
 - <https://scikit-learn.org/stable/modules/generated/sklearn.ensemble.RandomForestRegressor.html>
- Compared to R `randomForest()`:
 - Python used default `B = 100`. Please overwrite to 500.
 - Python used default RSF size = `M`. Please overwrite to `int(M/3)`.

【随机森林与 xgboost 的区别】

对比点	随机森林 (RF)	XGBoost
集成方式	Bagging (并行)	Boosting (串行, 逐步修正)
偏差/方差	偏差高, 方差低 (主要减少方差 (Variance), 避免过拟合)	偏差低, 方差可能高 (主要减少偏差 (Bias), 提高拟合能力。)
并行性	完全并行 (多棵树独立训练)	部分可并行, 但有依赖 (逐棵生成, 每一棵树拟合上一棵的残差)
正则化	无显式正则	内置 L1/L2 正则 (内置正则化项 (L1、L2), 防止过拟合)
超参数	少, 易调	多, 需仔细调参
特征重要性	基尼/信息增益 (通过 基尼指数/信息增益的平均下降 来评估特征重要性。)	Gain、Cover、Frequency (支持多种重要性度量 (Gain、Cover、Frequency), 解释性更丰富)
应用场景	baseline、快速建模、解释性强	高精度预测、Kaggle 竞赛、金融风控

RF = 稳健、简单、抗过拟合 → 适合作 baseline / 快速原型。

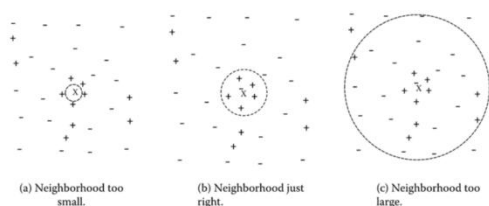
XGBoost = 灵活、精度高, 但要调参 → 适合追求极致性能的业务场景。

五、KNN (K Nearest Neighbor, K 近邻算法)

【总结】

- 找到距离输入点最近的 K 个数据点, 并根据这 K 个邻居中最常见的类别来对输入点进行分类。
- 常用于推荐系统, 根据用户的购买历史或偏好向其推荐产品。
- 不适用于稀疏数据 (sparse data)。

K-parameter



Source: Steinbach, M., & Tan, P. N. (2009). kNN: k-nearest neighbors. *The top ten algorithms in data mining*, 151-162.

5.1 基本思想

直观理解：一个样本的新标签，由它周围最近的 K 个已知样本的标签来决定。

- 分类任务：取 K 个邻居中出现最多的类别，作为预测结果。
- 回归任务：取 K 个邻居数值的平均值，作为预测结果。

5.2 工作流程

- 选择参数 K（比如 K=3 或 K=5）。
- 计算距离（常用欧式距离、曼哈顿距离、余弦相似度）。

🔪 1. 距离公式 (Distance Metrics)

KNN 的核心是“谁近谁远”，所以关键公式是“距离度量”：

(1) 欧式距离（最常见）

$$d(x_i, x_j) = \sqrt{\sum_{k=1}^n (x_{ik} - x_{jk})^2}$$

(2) 曼哈顿距离

$$d(x_i, x_j) = \sum_{k=1}^n |x_{ik} - x_{jk}|$$

(3) 闵可夫斯基距离（推广公式）

$$d(x_i, x_j) = \left(\sum_{k=1}^n |x_{ik} - x_{jk}|^p \right)^{\frac{1}{p}}$$

- 当 $p = 1 \rightarrow$ 曼哈顿距离
- 当 $p = 2 \rightarrow$ 欧式距离

(4) 余弦相似度（文本/推荐常用）

$$\text{sim}(x_i, x_j) = \frac{x_i \cdot x_j}{||x_i|| \cdot ||x_j||}$$

• 常用在文本、用户兴趣向量 $\rightarrow$ KNN 里可用“1 - 相似度”作为距离。

- 找出最近的 K 个邻居。
- 投票/平均：
 - 分类 $\rightarrow$ 采用多数表决法。
 - 回归 $\rightarrow$ 取均值或加权均值。

【例】

假设我们有以下训练数据（带标签）：

身高 (cm)	体重 (kg)	性别
170	65	男
180	80	男
160	50	女
155	48	女
165	55	女

现在给一个新样本：

$$x_{new} = (172, 70)$$

问：预测性别？



1、计算距离（公式部分）

1. 距离公式（常用欧式距离）

$$d(x_i, x_j) = \sqrt{\sum_{k=1}^n (x_{ik} - x_{jk})^2}$$

对新样本 (172, 70)，计算与每个点的距离：

• 与 (170,65,男)：

$$d = \sqrt{(172 - 170)^2 + (70 - 65)^2} = \sqrt{4 + 25} = \sqrt{29} \approx 5.39$$

• 与 (180,80,男)：

$$d = \sqrt{(172 - 180)^2 + (70 - 80)^2} = \sqrt{64 + 100} = \sqrt{164} \approx 12.81$$

• 与 (160,50,女)：

$$d = \sqrt{(172 - 160)^2 + (70 - 50)^2} = \sqrt{144 + 400} = \sqrt{544} \approx 23.32$$

• 与 (155,48,女)：

$$d = \sqrt{(172 - 155)^2 + (70 - 48)^2} = \sqrt{289 + 484} = \sqrt{773} \approx 27.80$$

• 与 (165,55,女)：

$$d = \sqrt{(172 - 165)^2 + (70 - 55)^2} = \sqrt{49 + 225} = \sqrt{274} \approx 16.55$$

2. 选取 K 个邻居

假设 K=3，则最近的 3 个邻居是：

(170,65,男) → d=5.39

(180,80,男) → d=12.81

(165,55,女) → d=16.55

3、投票决策

- 男：2 票
- 女：1 票

因此预测结果：

$$y_{new} = \text{男}$$

总结

- 核心公式：欧式距离

$$d(x_i, x_j) = \sqrt{\sum_{k=1}^n (x_{ik} - x_{jk})^2}$$

- 流程：计算距离 → 选最近的 K 个邻居 → 多数投票/取均值 → 输出预测。
- 结果：样本 (172,70) 被预测为 男性。

5.3 应用场景及优缺点

【应用场景】

推荐系统：根据用户的购买历史或偏好，找相似用户 → 推荐产品。

文本分类：根据与已知文档的相似度来分类。

模式识别：如手写数字识别（MNIST 数据集的早期经典方法）。

【优点】

简单直观，容易理解和实现。

无需训练过程 → “懒惰学习 (Lazy Learning)”。

适合多分类问题。

【缺点】

计算量大：预测时必须计算与所有训练样本的距离。

对噪声敏感：异常点可能干扰结果。

维度灾难：在高维稀疏数据中 (sparse data)，距离度量失效，效果差。

5.4 【总结】

- 找到距离输入点最近的 K 个数据点，并根据这 K 个邻居中最常见的类别来对输入点进行分类。
- 常用于推荐系统，根据用户的购买历史或偏好向其推荐产品。
- 不适用于稀疏数据 (sparse data)。

六、关联性规则

6.1 背景

【背景】

最初用于分析零售交易（即“购物篮分析”），目的在于：

哪些商品组合是受欢迎的？

向顾客推荐产品。

重新设计商店货架以优化零售销售

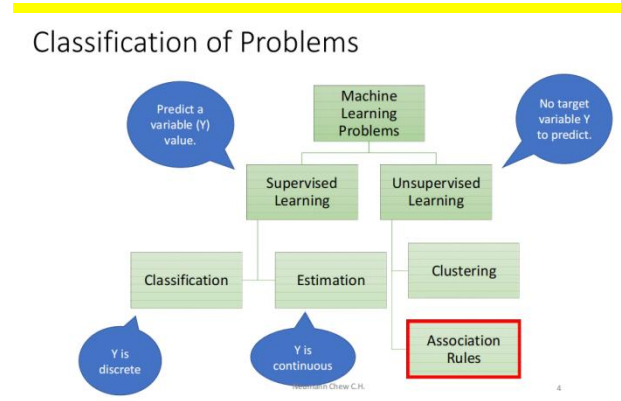
【现代应用（推荐系统）】

Amazon.com（电商）

淘宝

Netflix
Spotify
大多数推荐系统

【机器学习问题分类】



【Data Format 数据格式】

目的是找到哪些商品组合受欢迎。

需要交易型数据（类似收据）。

每条记录代表一次完整交易。

● 示例：NTUC 超市收据、手机合约发票、软件购买账单、亚马逊账单。

两种常见数据存储格式：

- 宽格式 (Wide Data Format)
- 长格式 (Long Data Format)

【Example of Wide Data Format 宽格式示例】

Example database with 5 transactions and 5 items

transaction ID	milk	bread	butter	beer	diapers
1	1	1	0	0	0
2	0	0	1	0	0
3	0	0	0	1	1
4	1	1	1	0	0
5	0	1	0	0	0

Source: Wikipedia.

- 每一行是一条交易记录。
- 每一列代表一个独特的商品。
- 每一行记录该交易是否购买某个商品（0 或 1）。
- 如果商品种类很多，数据会非常稀疏（大部分是 0）。

【Example of Long Data Format 长格式示例】

	A	B
1	ID	Item
2	1	Milk
3	4	Milk
4	1	Bread
5	4	Bread
6	5	Bread
7	2	Butter
8	4	Butter
9	3	Beer
10	3	Diapers

- 与上一页同一份数据集。
- 每一行仅包含交易 ID 和一件商品。
- 对分析有不同影响。

6.2 关联规则

【Fundamental Concepts 基本概念】

- 项集 (Itemset)
- 一次交易中的一组商品。
- 项集大小为 3，表示集合里有 3 个商品。
- 可以是任意大小。

【关联规则 (Association Rule)】

- 符号： $X \rightarrow Y$
- 表示 X 与 Y 相关。
- X 为“前件” (antecedent)，Y 为“后件” (consequent)。
- X 或 Y 都可能包含多个商品。

【3 Key Concepts 三个关键概念】

教科书和网站（如 Wikipedia）一般用“集合论”来解释支持度、置信度、提升度。其实用“概率”的方式更容易理解。

3 Key Concepts in Association Rules 关联规则中的三个概念

定义如下：

- 前件支持度 (Antecedent Support) : $\text{Supp}(X) = P(\text{包含 } X)$
- 规则支持度 (Rule Support) : $\text{Supp}(X \text{ and } Y) = P(\text{同时包含 } X \text{ 和 } Y)$
- 规则置信度 (Rule Confidence) : $\text{Conf}(X \rightarrow Y) = P(\text{包含 } Y \mid \text{已包含 } X)$
- 规则提升度 (Rule Lift) : $\text{Lift}(X \rightarrow Y)$

注：在概率论中 X 与 Y 是事件，通常是交集。但在关联规则中，X 与 Y 代表的是项集，通常没有交集，实际上是并集。

【Simple Numerical Example 简单数值例子】

Example database with 5 transactions and 5 items

transaction ID	milk	bread	butter	beer	diapers
1	1	1	0	0	0
2	0	0	1	0	0
3	0	0	0	1	1
4	1	1	1	0	0
5	0	1	0	0	0

给定规则 R1:

- {牛奶, 面包} → 黄油

计算:

- $\text{Supp}(\{\text{牛奶}\})$
- $\text{Supp}(\{\text{牛奶}, \text{面包}\})$
- $\text{Supp}(\{\text{牛奶}, \text{面包}, \text{黄油}\})$
- 规则 R1 的置信度 Confidence of the rule R1
- 规则 R1 的提升度 Lift of the rule R1

Simple Numerical Example (Answers) 简单数值例子 (答案)

Assoc Rule: {milk, bread} ——» Butter

$\text{Supp}(\{\text{牛奶}\}) = 2/5$

$\text{Supp}(\{\text{牛奶}, \text{面包}\}) = 2/5$

$\text{Supp}(\{\text{牛奶}, \text{面包}, \text{黄油}\}) = 1/5 = 0.2$

规则置信度 = $1/2 = 0.5$

提升度 = $(1/2) / (2/5) = 1.25$

关联规则: {牛奶, 面包} → 黄油

【Modern Applications (Non-Recommendation Sys) 现代应用 (非推荐系统)】

“商品”的定义可以扩展到任何事件，而不只是销售产品。

应用案例:

- 工作场所事故与危险
- 医学诊断
- 因此称为“关联规则”，而不只是“购物篮分析”。

【What is an Association Rule? 什么是关联规则】

一个关联规则是将一组商品与另一组商品联系起来。

符号: $X \rightarrow Y$

衡量规则有效性的三个关键指标:

- 支持度 (Support): 适用性

- 置信度 (Confidence)：预测强度
- 提升度 (Lift)：上下文（是否比随机更相关）

The Key Question 核心问题

在一个包含大量交易的数据库中，哪些商品组合是受欢迎的？

6.3 总结

Association Rules（关联规则）

目的：

快速、自动地识别有用的关联规则。

三大核心指标：

- Support（支持度）：某个规则在数据中出现的频率。
- Confidence（置信度）：在前件发生的情况下，后件发生的概率。
- Lift（提升度）：衡量规则的强度，表示前件和后件是否真的存在依赖关系。

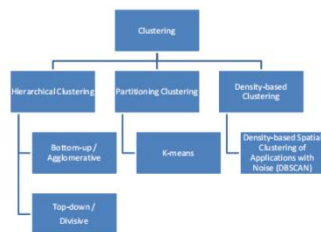
七、聚类分析

7.1 概述

无监督学习 - 聚类分析

- 聚类 (Clustering)：无监督机器学习方法。
- 有时也称为“分割”，即把数据划分为不同的部分。
- 无监督：自学习，没有固定输出或因变量。
- 主要思想：将项目分组到相似的簇中。
- 计算点与簇中心之间的距离，来决定合并或拆分。
- 过程持续，直到收敛（簇不再发生变化）。

Types of Clustering



【聚类的类型】

层次聚类 (Hierarchical Clustering)

- 自下而上 / 凝聚式 (Agglomerative)
- 自上而下 / 分裂式 (Divisive)

划分式聚类 (Partitioning Clustering)

- K-means

基于密度的聚类 (Density-based Clustering)

- 带噪声的基于密度空间聚类 (DBSCAN)

7.2 层次聚类

能够构建一个无需预先设定聚类数量的聚类层次结构。它通过以下其中一种方法生成树状结构：

【层次聚类 - 凝聚式 (Agglomerative)】

- 自下而上方法。
- 每个数据点一开始是一个独立簇。
- 将最近的簇合并为更大的簇。
- 持续进行直到形成一个簇。

【层次聚类 - 分裂式 (Divisive)】

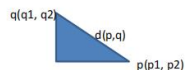
- 自上而下方法。
- 开始时把所有数据点放在一个簇中。
- 根据最远的距离逐步拆分成小簇。
- 重复，直到每个点都是独立簇。

7.3 K-means 聚类

- K = 簇的数量。
 - 例如 $K=4$ ，则需要预先确定或随机生成 4 个簇中心。
- 把每个点分配给最近的簇中心。
- 簇中心会根据簇内所有点的平均值更新。
- 迭代，直到簇不再变化。
- 可尝试不同的 K ，观察结果差异。

- Apart from configuring the value for k , we could also choose the distance measure to be used for distance calculation.
- The most commonly used distance measure is Euclidean Distance.
- Euclidean distance is calculated by taking the square difference between points then sum it up and square root. Analogous to Pythagorean theorem.
- Given point $p(p_1, p_2)$ and point $q(q_1, q_2)$, the two-dimensional Euclidean distance is given by:

$$d(p, q) = \sqrt{(q_1 - p_1)^2 + (q_2 - p_2)^2}$$



K-means 聚类 - 距离度量

- 除了配置 K ，还可以选择距离度量方法。
- 最常用的是 欧式距离：

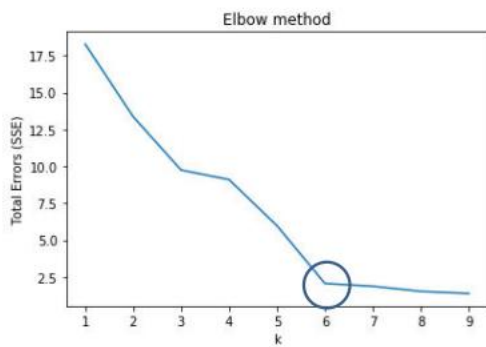
$$d(p, q) = \sqrt{(p_1 - q_1)^2 + (p_2 - q_2)^2}$$

- 类比勾股定理。

7.4 【K-means 与层次聚类对比】

K-means：计算复杂度较低；但不同初始中心可能得到不同结果。

层次聚类：结果一致，但凝聚式和分裂式结果不一定相同。



肘部法则 (Elbow Method)

- 选择合适簇数，当增加簇数不再显著改善分组时，停止。
- 计算 SSE (平方误差和)。
- 选择能保持 SSE 较低的小 K 值。

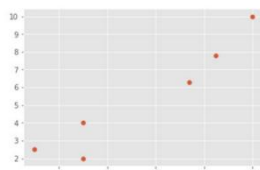
7.5 python 实现

【Scikit-learn 聚类】

- Python 的 sklearn.cluster 模块专门用于聚类。

【步骤】

- 初始化数据 → 转 numpy 格式 → 调用 KMeans() → 用 fit() 训练 → 得到 cluster_centers_ 和 labels_。



```
1 import numpy as np
2 import matplotlib.pyplot as plt
3 from sklearn.cluster import KMeans
4
5 # Initialize some data points to start with
6 X = np.array([[1, 2],
7               [1.5, 3.5],
8               [3, 4],
9               [4, 4],
10              [10, 6.5],
11              [10, 8],
12              [10, 9]])
13
14 # visualize original points on plot
15 plt.scatter(X[:,0],X[:,1],s=50)
16
17 # print out to see X values
18 print(X)
19
20 # initialize kmeans with 2 clusters
21 kmeans = KMeans(n_clusters=2)
22 # use data to do clustering
23 kmeans.fit(X)
24
25 # print out the coordinates of two cluster centers
26 print(kmeans.cluster_centers_)
27
28 # print out labels, i.e. each data point belongs to which cluster
29 print(kmeans.labels_)
30
31 # use each value according to the labels from kmeans
32 plt.scatter(X[:,0],X[:,1],c=kmeans.labels_, cmap='rainbow')
33
34 # plot out cluster centres with black colors
35 plt.scatter(kmeans.cluster_centers_[:,0],
36            kmeans.cluster_centers_[:,1], color='black')
```

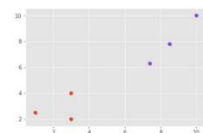
Example:

【KMeans 聚类绘图】

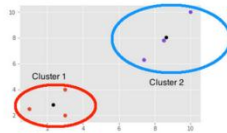
- 用不同颜色显示簇，中心点用黑色标注。

Kmeans clustering

- After fit(), plot using different colors (rainbow)



- Add in cluster centers in black.

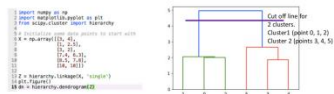


【层次聚类】

步骤：数据准备 → 聚类 → 选择簇数

Hierarchical Clustering

- There is a hierarchy clustering available from scipy module.
- See the codes below that generates the final Dendrogram on the right.



层次聚类 (Scipy 实现)

- Scipy 提供层次聚类函数。
- 可生成树状图 (Dendrogram) 。
- 例如：切分为 2 个簇 → 簇 1 (点 0,1,2) ，簇 2 (点 3,4,5) 。

7.6 例子

任务 T1 – 股票聚类

用股票数据 (stocks.csv) 做聚类，帮助投资组合优化。

投资者 1: Peter, 20 多岁，资金 10 万 SGD，愿意承担风险，3 年希望获得 50% 回报。

投资者 2: Mary, 60 多岁，资金 100 万 SGD，终身积蓄，希望低风险保值并有小额回报。

——» 要根据聚类结果给出投资建议。

Task T1 – Stocks Clustering

Clustering has wide applications. It can be used to form stock portfolios for effective investment strategy.

Given the stocks.csv file. Perform clustering on the data.

Two investors (Peter and Mary) seek your advice on investment. They have the following characteristics:

1. Peter: In his 20s with capital of SGD\$100,000. He is willing to risk a little and hope to get 50% return in 3 years time.
2. Mary: In her 60s with capital of \$1,000,000. This is her lifelong savings which she wishes to safeguard as much as possible so long as there are some returns.

How would you advise them on investment strategy?

Stock name	Last trade	PE	EPS	Payout Ratio	Dividend yield
Acandias Reit	2.57	15.9	0.2	93	5.91
Capitaland	3.07	9.4	0.3	37	3.9
Capitaland Commercial Trust	1.72	11.5	0.2	55	4.77
Capitaland Trust	2.11	11.7	0.2	61	5.27
City Developments	7.86	12.4	0.6	22	1.73

